

	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki Nowoczesne Technologie Przetwarzania Danych Laboratorium 13 Temat: Orkiestracja kontenerów w Kubernetes - wdrożenie API modelu ML na lokalnym klastrze	
---	---	---

Sprawozdanie, Marcin Gregrowicz, semestr VI, grupa nr 1

Cel ćwiczenia

Celem ćwiczenia jest praktyczne poznanie orkiestracji kontenerów w Kubernetes; uruchomienie lokalnego klastra; wdrożenie skonteneryzowanego API modelu ML (z wcześniejszych laboratoriów) jako Deployment i Service; skalowanie aplikacji; wykonanie aktualizacji kroczącej (rolling update) i wycofania zmian (rollback); skonfigurowanie sond zdrowia oraz limitów zasobów. Laboratorium pokazuje, jak przejść od pojedynczego kontenera Docker do zarządzanego, skalowalnego i samonaprawiającego się wdrożenia.

Link do repozytorium Git:

<https://github.com/MarGegr/NTPD13>

Zadanie 1: Przygotowanie środowiska

W tym laboratorium skorzystałem z narzędzia Minikube.

Wersja Kubernetesa:

```
(.venv) PS C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

Po uruchomieniu klastra działał on prawidłowo:

```
(.venv) PS C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13> kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready     control-plane  15m   v1.35.1
(.venv) PS C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:59403
CoreDNS is running at https://127.0.0.1:59403/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

Zadanie 2: Obraz kontenera modelu ML

Wykorzystałem API modelu ML z laboratorium 03, znajduje się w pliku app.py

Zbudowanie obrazu Dockera aplikacji:

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>docker build -t ml-api:1.0 .
[+] Building 2.5s (11/11) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 509B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 83B
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf1731b1b
=> [internal] load build context
=> => transferring context: 3.01kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:4706e4639a2852e0b8aa2653d9833b694272d259c696c0bd8525ee71c554c4b5
=> => exporting config sha256:c1a8f8bd376a0651f22642e9d2235d16289e9a0d4f726ed5e317d874d340f67d
=> => exporting attestation manifest sha256:002b1b461529d3d9bfe8c1b9b6f0b649aaa0776e55f2a065d39ae20c59538ac
=> => exporting manifest list sha256:6ab3b5690f65b23a5e25917302fc0930d752a2ac8fb29cb2922edb751b3e4a8c
=> => naming to docker.io/library/ml-api:1.0
=> => unpacking to docker.io/library/ml-api:1.0
```

Nazwa obrazu: ml-api

Tag obrazu: 1.0

Zadanie 3: Deployment i Service

Manifest Deployment znajduje się w pliku deployment.yaml

Manifest Service znajduje się w pliku service.yaml

Wyświetlenie uruchomionych podów:

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ml-api-85bccc8777-8nwdx             1/1     Running   0           16s
ml-api-85bccc8777-nxqkx             1/1     Running   0           16s
```

Wyświetlenie adresu URL, pod którym dostępna jest usługa:

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>minikube service ml-api --url
http://127.0.0.1:61615
! Z powodu użycia sterownika dockera na systemie operacyjnym windows, terminal musi zostać uruchomiony.
```

Zadanie 4: Skalowanie, aktualizacja krocząca i rollback

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ml-api-85bccc8777-5556q             1/1     Running   0           6s
ml-api-85bccc8777-8nwdx             1/1     Running   0          3m36s
ml-api-85bccc8777-nxqkx            1/1     Running   0          3m36s
ml-api-85bccc8777-trmgd            1/1     Running   0           6s
```

Przy zmianie liczby replik oraz przy usuwaniu klaster automatycznie powracał do zadanej liczby podów.

Zbudowanie nowej wersji obrazu:

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>minikube image load ml-api:2.0
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl set image deployment/ml-api ml-api=ml-api:2.0
deployment.apps/ml-api image updated

C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ml-api-5878457d8-4w7df             1/1     Running   0           14s
ml-api-5878457d8-7w98z             1/1     Running   0           17s
ml-api-5878457d8-h69vd             1/1     Running   0           17s
ml-api-5878457d8-xcqbv             1/1     Running   0           16s
ml-api-85bccc8777-nxqkx            1/1     Terminating 0          6m51s

C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl rollout status deployment/ml-api
deployment "ml-api" successfully rolled out

C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ml-api-5878457d8-4w7df             1/1     Running   0           41s
ml-api-5878457d8-7w98z             1/1     Running   0           44s
ml-api-5878457d8-h69vd             1/1     Running   0           44s
ml-api-5878457d8-xcqbv             1/1     Running   0           43s

C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl rollout undo deployment/ml-api
deployment.apps/ml-api rolled back

C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ml-api-85bccc8777-65j7v             1/1     Running   0           28s
ml-api-85bccc8777-h6qwf             1/1     Running   0           24s
ml-api-85bccc8777-jdmnn            1/1     Running   0           27s
ml-api-85bccc8777-v6fxm            1/1     Running   0           28s
```

Przy skalowaniu w górę Kubernetes natychmiast powołuje do życia nowe, identyczne pody, przydziela im adresy IP i podpiną pod serwis balansujący ruch. Przy skalowaniu w dół kontroler wybiera nadmiarowe pody, wysyła do nich sygnał bezpiecznego zamknięcia

Kubernetes domyślnie przeprowadza też aktualizację bezprzerwową. Stopniowo, po jednej lub kilka sztuk, tworzy nowe pody z zaktualizowaną wersją aplikacji, a gdy przejdą one testy gotowości (readiness), kieruje na nie ruch sieciowy i dopiero wtedy kasuje odpowiadającą im liczbę starych podów, aż do całkowitej wymiany całej puli.

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
ml-api-5878457d8-4w7df	1/1	Running	0	2m38s
ml-api-5878457d8-7w98z	1/1	Running	0	2m41s
ml-api-5878457d8-h69vd	1/1	Running	0	2m41s
ml-api-5878457d8-xcqbv	1/1	Running	0	2m40s
ml-api-85bccc8777-v6fxm	0/1	Pending	0	0s
ml-api-85bccc8777-v6fxm	0/1	Pending	0	0s
ml-api-5878457d8-4w7df	1/1	Terminating	0	2m58s
ml-api-85bccc8777-v6fxm	0/1	ContainerCreating	0	0s
ml-api-85bccc8777-65j7v	0/1	Pending	0	0s
ml-api-85bccc8777-65j7v	0/1	Pending	0	0s
ml-api-5878457d8-4w7df	1/1	Terminating	0	2m58s
ml-api-85bccc8777-65j7v	0/1	ContainerCreating	0	0s
ml-api-85bccc8777-v6fxm	1/1	Running	0	1s
ml-api-5878457d8-7w98z	1/1	Terminating	0	3m2s
ml-api-5878457d8-7w98z	1/1	Terminating	0	3m2s
ml-api-85bccc8777-jdmnn	0/1	Pending	0	0s
ml-api-85bccc8777-jdmnn	0/1	Pending	0	0s
ml-api-85bccc8777-jdmnn	0/1	ContainerCreating	0	1s
ml-api-5878457d8-4w7df	0/1	Completed	0	3m
ml-api-5878457d8-4w7df	0/1	Completed	0	3m1s
ml-api-5878457d8-4w7df	0/1	Completed	0	3m1s
ml-api-5878457d8-4w7df	0/1	Completed	0	3m1s
ml-api-85bccc8777-65j7v	1/1	Running	0	3s
ml-api-5878457d8-xcqbv	1/1	Terminating	0	3m4s
ml-api-5878457d8-xcqbv	1/1	Terminating	0	3m4s
ml-api-85bccc8777-h6qwf	0/1	Pending	0	1s
ml-api-85bccc8777-h6qwf	0/1	Pending	0	1s
ml-api-85bccc8777-h6qwf	0/1	ContainerCreating	0	1s
ml-api-5878457d8-7w98z	0/1	Completed	0	3m8s
ml-api-5878457d8-7w98z	0/1	Completed	0	3m10s
ml-api-5878457d8-7w98z	0/1	Completed	0	3m10s
ml-api-5878457d8-7w98z	0/1	Completed	0	3m10s
ml-api-5878457d8-xcqbv	0/1	Completed	0	3m10s
ml-api-5878457d8-xcqbv	0/1	Completed	0	3m11s
ml-api-5878457d8-xcqbv	0/1	Completed	0	3m12s
ml-api-5878457d8-xcqbv	0/1	Completed	0	3m12s
ml-api-85bccc8777-h6qwf	1/1	Running	0	9s
ml-api-85bccc8777-jdmnn	1/1	Running	0	13s
ml-api-5878457d8-h69vd	1/1	Terminating	0	3m15s
ml-api-5878457d8-h69vd	1/1	Terminating	0	3m15s
ml-api-5878457d8-h69vd	0/1	Completed	0	3m21s
ml-api-5878457d8-h69vd	0/1	Completed	0	3m22s
ml-api-5878457d8-h69vd	0/1	Completed	0	3m22s
ml-api-5878457d8-h69vd	0/1	Completed	0	3m22s

Komenda `kubectl get pods -w` pokazuje historię zmian na podach. Podczas aktualizacji kroczącej nigdy nie zostały zatrzymane naraz wszystkie pody, aktualizacja była wykonywana po jednym podzie, co zapewniło brak przerwy w działaniu aplikacji, więc API było cały czas dostępne.

Zadanie 5: Konfiguracja, sondy zdrowia i limity zasobów

Sondy, żądania i limity znajdują się w pliku `deployment.yaml`

Konfiguracja aplikacji została wyniesiona do `configmap.yaml`

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ml-api-85b48d7b6c-jb76c	1/1	Running	0	52s
ml-api-85b48d7b6c-rbsm5	1/1	Running	0	29s

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
ml-api	Deployment/ml-api	cpu: <unknown>/60%	2	6	2	58s

Skonfigurowałem również automatyczne skalowanie poziome, jak widać na zrzucie poniżej, liczba podów automatycznie zmieniła się (najpierw 4 pody, a następnie 6).

```
C:\Users\Marcin\Desktop\ARiSzCz\Lab\Kod\NTPD13\App>kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
ml-api	Deployment/ml-api	cpu: 473%/60%	2	6	4	103s
ml-api	Deployment/ml-api	cpu: 473%/60%	2	6	6	106s
ml-api	Deployment/ml-api	cpu: 427%/60%	2	6	6	2m2s

Pojedynczy kontener Docker a wdrożenie w Kubernetes

Pojedynczy kontener Docker to wyizolowana aplikacja uruchomiona na konkretnej, pojedynczej maszynie, wymagająca ręcznego zarządzania jej cyklem życia, siecią oraz pamięcią. Z kolei wdrożenie w Kubernetes przenosi aplikację w środowisko klastra wielu maszyn, automatyzując zadania takie jak skalowanie (dokładanie kolejnych kopii kontenera w razie wzrostu ruchu), automatyczne przywracanie działających wersji po awarii oraz zaawansowany routing sieciowy bez ingerencji administratora.

Podejście deklaratywne (YAML) a imperatywne (kubectl ad hoc)

Podejście imperatywne polega na wydawaniu pojedynczych, doraźnych poleceń (np. `kubectl run`), w których instruuje się system krok po kroku, jakie akcje ma w tym momencie wykonać, co jest szybkie przy testach, ale trudne do odtworzenia. Podejście deklaratywne opiera się na plikach manifestów YAML, w których szczegółowo opisuje się docelowy, oczekiwany stan infrastruktury. Kubernetes nieustannie dąży do jego osiągnięcia i utrzymania, co pozwala na pełną automatyzację i przechowywanie konfiguracji w systemach kontroli wersji.

Sonda readiness a sonda liveness

Sonda liveness odpowiada za sprawdzanie, czy aplikacja wewnątrz kontenera działa i czy nie zawiesiła się. W przypadku niepowodzenia tego testu Kubernetes natychmiast restartuje dany kontener. Sonda readiness weryfikuje natomiast, czy aplikacja jest już w pełni gotowa na przyjmowanie ruchu sieciowego (np. czy zdążyła połączyć się z bazą danych i załadować pamięć podręczną). Jeśli ten test zawiedzie, Kubernetes nie restartuje kontenera, a jedynie tymczasowo odłącza go od load balancera, aby użytkownicy nie natrafiali na nie działającą stronę.